

P8X Programmer's Guide

Instruction set rev 1 - generated from the microcode source (genucode.py); opcode values, byte and cycle counts are extracted from the same tables that build the EPROM images, so this document cannot drift from the hardware.

Programming model

A, B - 8-bit ALU operand registers; results land in A. **P0-P3** - four 16-bit pointer registers; the address bus is always driven by one of them. **P0** is the program counter. **P3** is the stack pointer (empty-descending: push writes then decrements; initialise it early - see note 3). P1 and P2 are general pointers used by the (Pn) addressing modes. **FLAGS** - C, Z, N, V, latched only by instructions marked in the Flags column.

Memory map (rev E): \$0000-\$1FFF ROM (8K) | \$2000-\$FEFF RAM (56K) | \$FF00-\$FFFF I/O: \$FF00 switches (r), \$FF02 LEDs (w), \$FF04 ACIA status (bit0 RX ready, bit1 TX ready), \$FF05 ACIA data.

Reset: P0 is forced to \$0000; execution begins there. All other registers (including P3) are undefined on real hardware.

Instruction set

Op	Mnemonic	Bytes	Cycles	Flags	Description
System					
\$00	NOP	1	2	-	No operation.
\$01	HLT	1	2	-	Halt the clock. Resume only by reset (or emulator exit).
\$72	CLC	1	2	C	C := 0.
\$73	SEC	1	2	C	C := 1.
Interrupts (rev C)					
\$02	EI	1	2	-	Enable maskable interrupts (IE := 1).
\$03	DI	1	2	-	Disable maskable interrupts (IE := 0).
\$04	RTI	1	10	-	Return from interrupt: pop flags then PC; re-enables IE.
\$08	IRQ	1	10	-	Software interrupt: push PC+flags, vector to \$0808. Also the opcode the hardware forcing buffer injects on a maskable IRQ.
Load / store					
\$10	LDA #imm	2	2	-	A := immediate byte.
\$11	LDB #imm	2	2	-	B := immediate byte.
\$12	LDA addr	3	6	Z N	A := byte at addr (absolute).
\$13	LDB addr	3	6	Z N	B := byte at addr (absolute).
\$14	STA addr	3	6	-	Byte at addr := A (absolute).
\$15	LDA (P1)+	1	2	-	A := memory at P1, then P1 := P1 + 1.
\$16	LDA (P2)+	1	2	-	A := memory at P2, then P2 := P2 + 1.
\$17	LDA (P3)+	1	2	-	A := memory at P3, then P3 := P3 + 1.
\$19	STA (P1)+	1	2	-	Memory at P1 := A, then P1 := P1 + 1.
\$1A	STA (P2)+	1	2	-	Memory at P2 := A, then P2 := P2 + 1.
\$1B	STA (P3)+	1	2	-	Memory at P3 := A, then P3 := P3 + 1.
\$1D	STA (P1)	1	2	-	Memory at P1 := A.
\$1E	STA (P2)	1	2	-	Memory at P2 := A.
\$1F	STA (P3)	1	2	-	Memory at P3 := A.
\$51	LDA (P1)	1	2	Z N	A := memory at P1 (P1 unchanged).
\$52	LDA (P2)	1	2	Z N	A := memory at P2 (P2 unchanged).
\$53	LDA (P3)	1	2	Z N	A := memory at P3 (P3 unchanged).
ALU (operands A,B; result to A unless noted)					
\$20	ADD	1	2	C Z N	A := A + B.

Op	Mnemonic	Bytes	Cycles	Flags	Description
\$21	SUB	1	2	C Z N	$A := A - B.$
\$22	AND	1	2	C Z N	$A := A \text{ AND } B.$
\$23	OR	1	2	C Z N	$A := A \text{ OR } B.$
\$24	XOR	1	2	C Z N	$A := A \text{ XOR } B.$
\$25	CMP	1	2	C Z N	Flags from $A - B$; A unchanged.
\$26	INC	1	2	C Z N	$A := A + 1.$ (B not used.)
\$27	DEC	1	2	C Z N	$A := A - 1.$ (B not used.)
\$28	SHL	1	2	C Z N	$A := A \ll 1$, 0 into bit 0. (1)
\$29	SHR	1	2	C Z N	$A := A \gg 1$, 0 into bit 7. (1)
\$2A	ROL	1	2	C Z N	Rotate A left through carry.
\$2B	ROR	1	2	C Z N	Rotate A right through carry.
ALU with T (rev C; 2nd operand = T register via B-mux; B preserved)					
\$80	ADDT	1	2	C Z N	$A := A + T.$ (B preserved.)
\$81	SUBT	1	2	C Z N	$A := A - T.$ (B preserved.)
\$82	ANDT	1	2	C Z N	$A := A \text{ AND } T.$ (B preserved.)
\$83	ORT	1	2	C Z N	$A := A \text{ OR } T.$ (B preserved.)
\$84	XORT	1	2	C Z N	$A := A \text{ XOR } T.$ (B preserved.)
\$85	CMPT	1	2	C Z N	Flags from $A - T$; A and B unchanged.
\$86	LDT #imm	2	2	-	$T :=$ immediate byte. (No flags.)
\$87	LDT addr	3	6	-	$T :=$ byte at addr (absolute). (No flags.)
Pointer registers					
\$31	LPL1 #imm	2	3	-	Low byte of $P1 :=$ immediate.
\$32	LPL2 #imm	2	3	-	Low byte of $P2 :=$ immediate.
\$33	LPL3 #imm	2	3	-	Low byte of $P3 :=$ immediate.
\$35	LPH1 #imm	2	3	-	High byte of $P1 :=$ immediate.
\$36	LPH2 #imm	2	3	-	High byte of $P2 :=$ immediate.
\$37	LPH3 #imm	2	3	-	High byte of $P3 :=$ immediate.
\$54	INP1	1	2	-	$P1 := P1 + 1.$
\$55	INP2	1	2	-	$P2 := P2 + 1.$
\$56	INP3	1	2	-	$P3 := P3 + 1.$
\$58	DEP1	1	2	-	$P1 := P1 - 1.$
\$59	DEP2	1	2	-	$P2 := P2 - 1.$
\$5A	DEP3	1	2	-	$P3 := P3 - 1.$
\$5E	TAP1L	1	2	-	Low byte of $P1 := A.$
\$5F	TAP1H	1	2	-	High byte of $P1 := A.$
\$60	TAP2L	1	2	-	Low byte of $P2 := A.$
\$61	TAP2H	1	2	-	High byte of $P2 := A.$
\$62	TAP3L	1	2	-	Low byte of $P3 := A.$
\$63	TAP3H	1	2	-	High byte of $P3 := A.$
\$68	TPA1L	1	2	Z N	$A :=$ low byte of $P1.$
\$69	TPA1H	1	2	Z N	$A :=$ high byte of $P1.$
\$6A	TPA2L	1	2	Z N	$A :=$ low byte of $P2.$

Op	Mnemonic	Bytes	Cycles	Flags	Description
\$6B	TPA2H	1	2	Z N	A := high byte of P2.
\$6C	TPA3L	1	2	Z N	A := low byte of P3.
\$6D	TPA3H	1	2	Z N	A := high byte of P3.
Stack					
\$70	PHA	1	2	-	Push A onto the P3 stack.
\$71	PLA	1	3	Z N	Pop A from the P3 stack.
16-bit memory ops (rev D; compiler space savers). PHW/PLW/LPW pure-microcode; MOVW adds the PT2 scratch pointer					
\$74	PHW addr	3	10	-	Push the 16-bit word at addr onto the P3 stack (low byte first, high on top).
\$75	PLW addr	3	12	-	Pop a 16-bit word from the P3 stack into addr (high then low).
\$76	LPW1 addr	3	9	-	P1 (16-bit) := the word at addr.
\$77	LPW2 addr	3	9	-	P2 (16-bit) := the word at addr.
\$78	MOVW dst,src	5	13	-	16-bit memory->memory move: the word at src -> dst (via the PT/PT2 scratch pointers).
Control flow					
\$40	JMP addr	3	5	-	P0 (PC) := addr.
\$41	JSR (P1)	1	9	-	Push return address (high byte first) onto P3 stack, then P0 := P1. Target must already be in P1.
\$42	RTS	1	7	-	Pop return address from P3 stack into P0.
\$43	JSR addr	3	13	-	Push return address, then P0 := addr (absolute call).
\$48	BZ addr	3	5	-	Branch to addr if Z=1.
\$49	BNZ addr	3	5	-	Branch to addr if Z=0.
\$4A	BCP addr	3	5	-	Branch if C=1, i.e. the RAW 74181 Cn+4 pin is high. Pin high means NO carry out - see note (2).
\$4C	JNC addr	3	5	-	Branch to addr if C=0. (JC/JZ/JNZ are aliases of BCP/BZ/BNZ.)
Signed branches (rev C; after CMP — N^V/Z)					
\$44	BLT addr	3	5	-	Branch if signed A < B (N^V=1). Use after CMP.
\$45	BGE addr	3	5	-	Branch if signed A >= B (N^V=0). Use after CMP.
\$46	BLE addr	3	5	-	Branch if signed A <= B ((N^V) Z). Use after CMP.
\$47	BGT addr	3	5	-	Branch if signed A > B (not (N^V) Z). Use after CMP.
-	LDPn #imm16	4	6	-	Assembler pseudo-op: LPLn + LPHn pair. Pn := imm16.

Notes

(1) Shifts & rotates: SHL/SHR shift in 0 and latch the shifted-out bit into C. ROL/ROR rotate through C (the shifted-in bit is the current C). This makes multi-byte shifts work the conventional way (SHL low byte, then ROL high byte).

(2) C flag (rev B): CONVENTIONAL active-high carry. After ADD, C=1 means a carry occurred; after SUB/CMP, C=1 means no borrow (A >= B). JC/BCP branch on C=1, JNC on C=0. CLC/SEC clear/set C without disturbing Z/N/V. (Rev A latched the raw active-low 74181 Cn+4 pin; rev B embraces the conventional convention.)

(3) Stack: JSR pushes the return address high byte then low byte, decrementing after each write (empty-descending). RTS increments then reads. Software must initialise P3 (e.g. LDP3 #\$FEFF) before the first JSR. **(4) V flag:** reads 0 in rev A. **(5) Absolute addressing:** LDA/LDB/STA/JSR accept an absolute address; the hardware forms it in the hidden PT scratch pointer.

Assembler quick reference (p8xasm.py)

```
label:  MNEMONIC operand          ; comment
operands:  #expr (immediate) | (Pn) / (Pn)+ | expr (16-bit address)
exprs:     $1F 0x1F 31 'c' symbol with +/-, <expr = low byte, >expr = high
directives: .org e .byte e,... .word e,... .ascii "s" .asciiz "s"
```

```

        .fill n[,v]      NAME = expr      .equ NAME, expr
pseudo-op: LDPn #imm16      ; expands to LPLn #<imm, LPHn #>imm
usage:      python3 p8xasm.py prog.asm -o eeprom.bin [-l prog.lst]

```

Example: print a string

```

ACIA_D = $FF05
        .org 0
        LDP2 #ACIA_D      ; P2 -> ACIA data register
        LDP1 #msg         ; P1 -> string
        LDB #0
loop:   LDA (P1)+          ; fetch byte, advance
        OR                ; A := A|0 - sets Z on the terminator
        BZ done
        STA (P2)          ; transmit
        JMP loop
done:   HLT
msg:    .ascii "P8X lives!\r\n"

```

Writing a program for P8X/OS

Programs launched by the OS **RUN** command load into the transient program area at **\$6A00** and run via a **JSR** to their exec address. The program ABI: **return to the shell with RTS** (P3, the stack, is the OS's - leave it balanced); on entry **P2 points at the argument tail** - the command text after the program name, NUL-terminated (so `RUN EDIT F00.ASM` enters with P2 -> "F00.ASM"); programs that take no arguments ignore P2. Build with `.org $6A00` and the host assembler's `--base 0x6A00`, or assemble on-target with ASM. A file created on-target carries load/exec 0, which the OS maps to \$6A00, so it is directly RUNnable. The BIOS jump table at \$0100 (console + CF, the FFIND/FCREATE/FDELETE/FCOMMIT file calls, the FOPEN/FGETB and FWOPEN/FPUTB/FCLOSE byte streams, and FRESOLVE/FNORM/FOPENDIR/FNEXT) is the only entry point a program needs - it must not call into OS internals.